

Python 数值分析 算法实践



Python Numerical Analysis Algorithm Practice

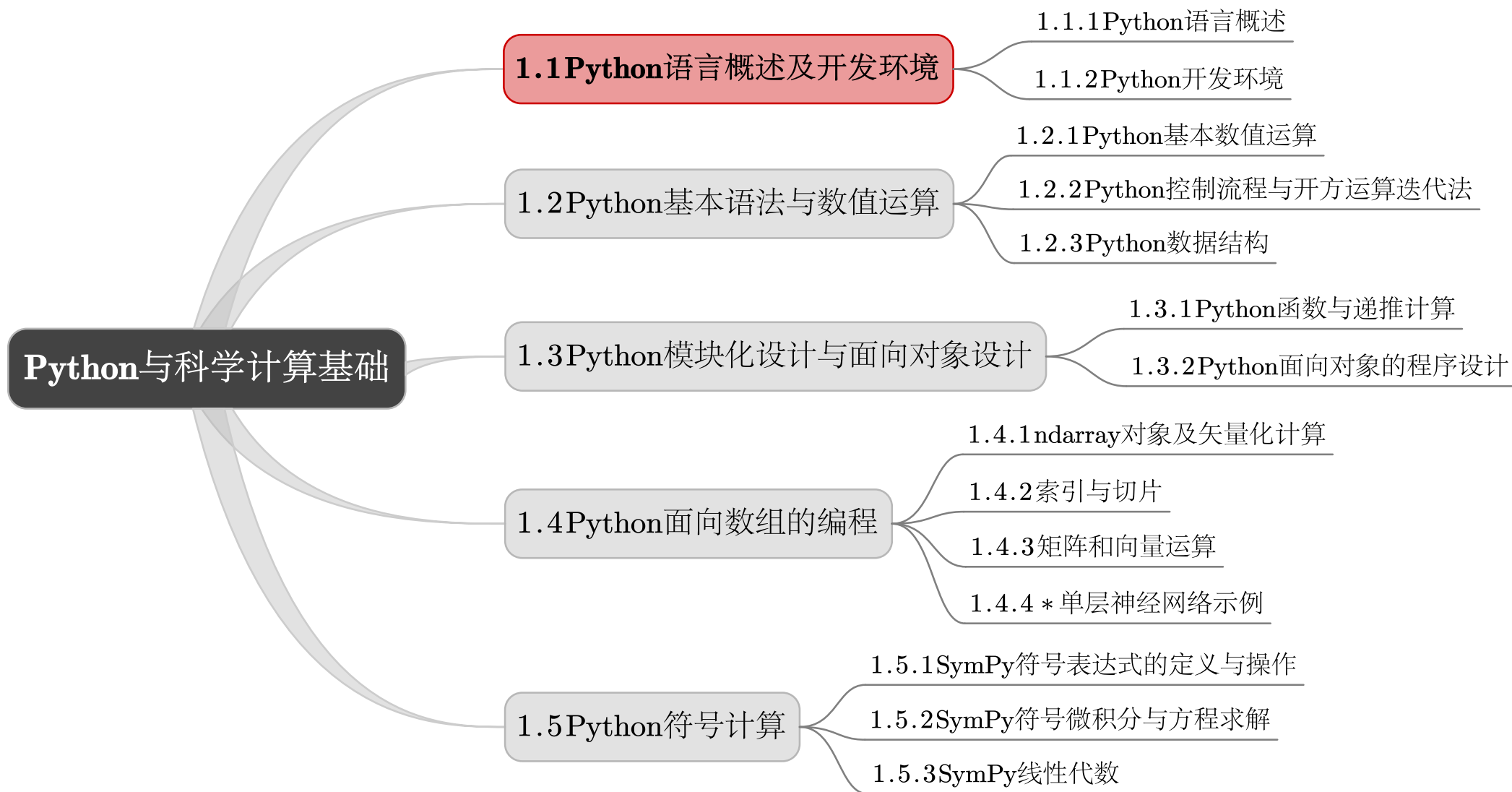
第1章 Python与科学计算基础



讲授: X X X



日期: 2024年4月19日



1.1.1 Python语言概述

表 1-1 Python 中常用库及其说明

标准库名	说明
numpy	NumPy 是一个在 Python 中做科学计算的基础库, 重在数值计算. 换句话说, NumPy 是大部分 Python 科学计算库的基础库, 多用于在大型、多维数组上执行数值运算.
scipy	SciPy 是 Python 一个著名的、基于 NumPy 之上开源科学库, SciPy 一般都是操纵 NumPy 数组来进行科学计算、统计分析. SciPy 提供了许多科学计算的库或子模块, 如线性代数 linalg、积分 integrate、优化 optimize、统计 stats 等.
pandas	Pandas 是 Python 的数据分析库. Pandas 在 NumPy 基础上补充了很多对数据处理特别有用的功能, 如标签索引、分层索引、数据对齐、合并数据集合、处理丢失数据等. 因此, Pandas 库已成为 Python 中执行高级数据处理的事实标准库, 尤其适用于统计应用分析.
matplotlib	Matplotlib 是 Python 最主要的图形可视化库, 用于生成静态的、达到出版质量的 2D 和 3D 图形, 支持不同的输出格式. 可视化是计算研究领域非常重要的组成部分. 此外, Seaborn 库是基于 Matplotlib 的针对统计数据分析的高级绘图库.
sympy	SymPy 是 Python 的一个数学符号计算库, 即符号计算. 它的目的在于成为一个富有特色的计算机代数系统, 它保证自身的代码尽可能地简单, 且易于理解, 容易扩展. SymPy 完全由 Python 写成, 不需要额外的库.
scikit-learn	scikit-learn 是 Python 中最著名也是最全面的机器学习库, 常见的算法如决策树、支持向量机、集成方法、聚类、神经网络等.

1.1.2 Python开发环境

The screenshot displays the PyCharm IDE interface. The top menu bar includes File, Edit, View, Navigate, Code, Refactor, Run, Tools, VCS, Window, and Help. The title bar shows the project name 'NumericalCalculationMethod - genetic_algorithm.py'.

On the left, the 'Project' tool window shows the project structure:

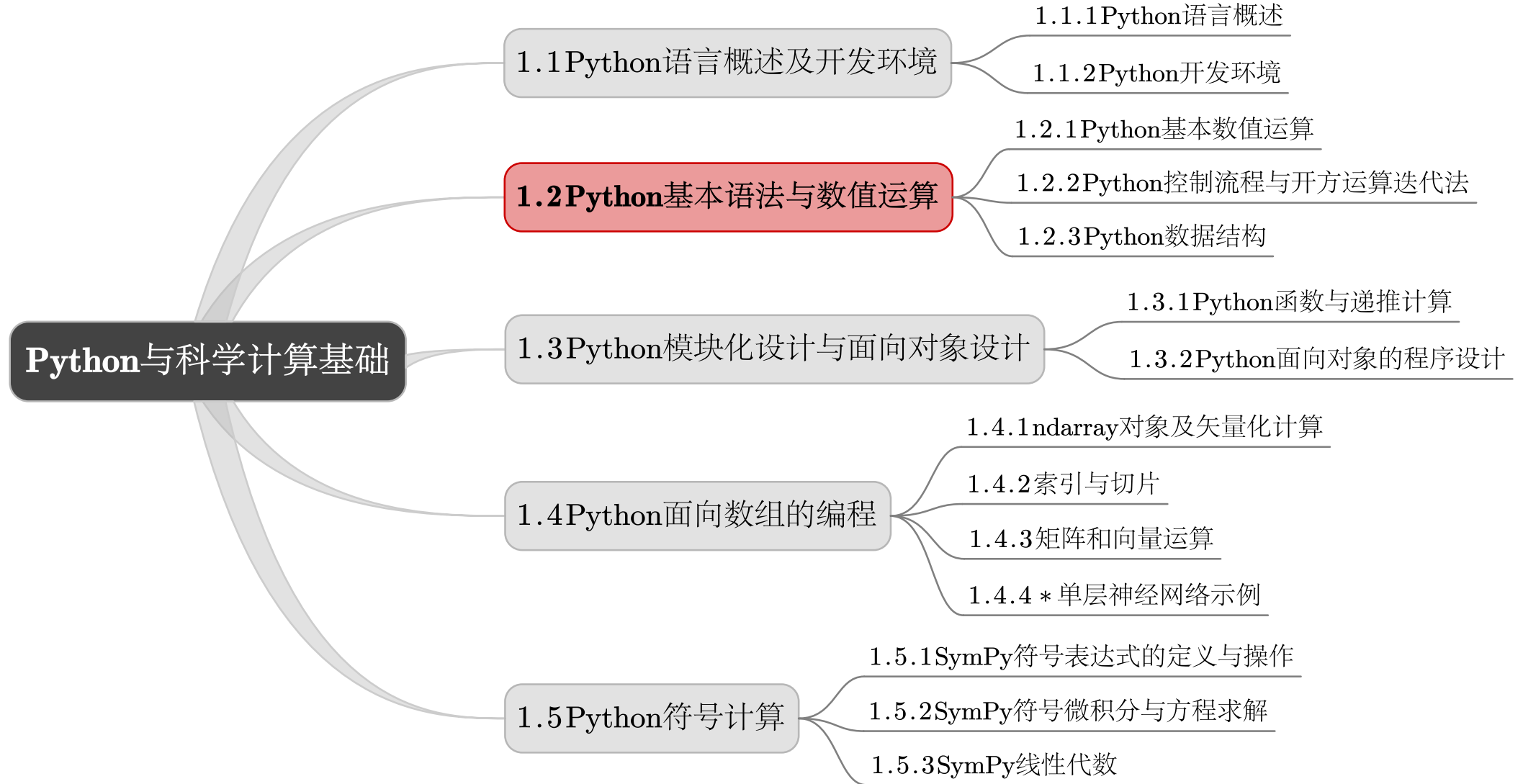
- Project: NumericalCalculationMethod (D:\demo\py...)
- Sub-packages:
 - direct_solution_linear_equations_06
 - function_approximation_03
 - fundamentals_python_mathematics_01
 - interpolation_02
 - iterative_solution_linear_equation_07
 - matrix_eigenvalue_calculation_10
 - nonlinear_equations_09
 - numerical_differentiation_05
 - numerical_integration_04
 - numerical_optimization_13
 - test
- Python modules (under numerical_optimization_13):
 - __init__.py
 - ant_colony_algorithm.py
 - bfgs_quasi_newton.py
 - cubic_approximation.py
 - dfp_quasi_newton.py
 - fibonacci_search.py
 - genetic_algorithm.py

On the right, the 'Code' tool window shows the code for 'genetic_algorithm.py':

```
1  # -*- coding: UTF-8 -*-
2  """
3  @file_name: genetic_algorithm.py
4  @time: 2022-08-18
5  @IDE: PyCharm Python: 3.9.7
6  @copyright: http://maths.xynu.edu.cn
7  """
8  import numpy as np
9  from decimal import Decimal
10
11  常用命令：一键格式化代码规范Reformat Code(Ctrl + Alt + L)
12  class GeneticAlgorithmOptimization:
13      """
14      遗传算法求解函数的最优值，默认求最大值，最小值则需在目标函数之前添加
15      """
16
17      def __init__(self, func, args_span, args_precision, pop_size=50,
18                  p_mutation=0.01, min_epochs=50, max_epochs=1000, ep:
19                  self.func = func # 待优化的函数, 适应度
20                  self.args_span = np.asarray(args_span) # 求解区间, 格式为[[x
21                  self.args_precision = np.asarray(args_precision, np.int) #
```

Annotations with red lines pointing to specific elements:

- 项目名称 Project: Points to the 'Project' tool window header.
- 各章包名 Package: Points to the 'NumericalCalculationMethod' folder in the project structure.
- 各章节内 Python 算法模块: Points to the 'genetic_algorithm.py' file in the 'numerical_optimization_13' sub-package.
- Python算法模块名称: Points to the '@file_name: genetic_algorithm.py' comment in the code.
- 当前算法编辑窗口: Points to the 'genetic_algorithm.py' tab in the code editor.



1.2.1 Python基本数值运算

表 1-2 Python 中运算符优先级及其描述

运算符	描述
**	指数 (最高优先级)
~, +, -	表示按位翻转, 一元加号和减号 (最后两个的方法名为 +@ 和 -@)
*, /, %, //	乘、除、取模和取整除
+, -	加法、减法
>>, <<	右移、左移运算符
&	按位与运算符
^,	位运算符, 分别表示按位异或、按位或
<=, <, >, >=	比较运算符, 分别表示小于等于、小于、大于、大于等于
==, <>, !=	等于运算符 ==, 不等于运算符 <> 与!= 含义相同
=, %=, /=, //=, -=, +=, *=, **=	赋值运算符 (先运算后赋值)
is, is not	身份运算符, 用于比较两个对象的存储单元 (同一块内存空间), 而 == 或!= 用于判断引用变量的值是否相等或不相等.
in, not in	成员运算符, 表示在指定的序列中查找值
not, and, or	逻辑运算符, 分别表示布尔 “与”、布尔 “或”、布尔 “非”

1.2.1 Python基本数值运算

表 1-3 math 库常见的函数和常量及其说明

函数或常量	数学表示说明	函数	数学表示说明	函数	数学表示说明
pi, e	π, e^1	sqrt(x)	$\sqrt{x}$	asin(x), acos(x)	$\arcsin x, \arccos x$
hypot(x, y)	$\sqrt{x^2 + y^2}$	exp(x)	e^x	atan(x)	$\arctan x$
ceil(x), floor(x)	$\lceil x \rceil, \lfloor x \rfloor$	degrees(x)	$180^\circ / \pi \times x$	fabs(x)	$ x $
pow(x, y)	x^y	radians(x)	$\pi / 180^\circ \times x$	factorial(x)	$x!$
log(x), log1p(x)	$\ln x, \ln(1 + x)$	sin(x), cos(x)	$\sin x, \cos x$	fsum(X)	若 X 为可迭代对象, 且元素为数值, 则返回 $x_1 + x_2 + \cdots x_n$
log10(x), log2(x)	$\lg x, \log_2 x$	tan(x)	$\tan x$	trunc(x)	x 的整数部分
modf(x)	提取 x 小数和 整数组成元组	isinf(x)	判断 x 是否为 无穷或负无穷	isnan(x)	判断 x 是否为 NaN (not a number)

1.2.2 Python控制流程与开方运算迭代法

迭代法是一种按同一公式重复计算逐次逼近真值的算法, 是数值计算普遍使用的重要方法. 迭代计算必然采用循环结构, 常常配合逼近精度 $\varepsilon > 0$ 和最大迭代次数 $k > 0$ 使用. 选择结构可在循环体内, 计算每次近似值(或近似解) x^* 的精度, 若满足则终止迭代.

设要开方的数为 $a \geq 0$, 开方运算求解 $\sqrt{a}$ 的迭代公式为

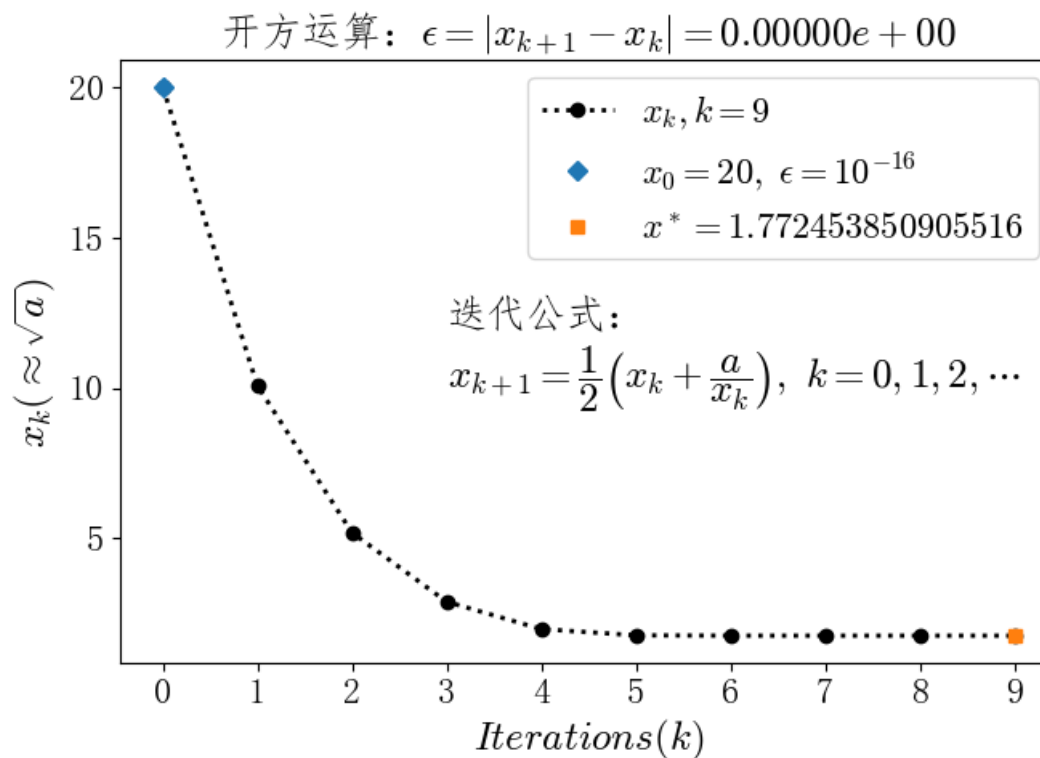
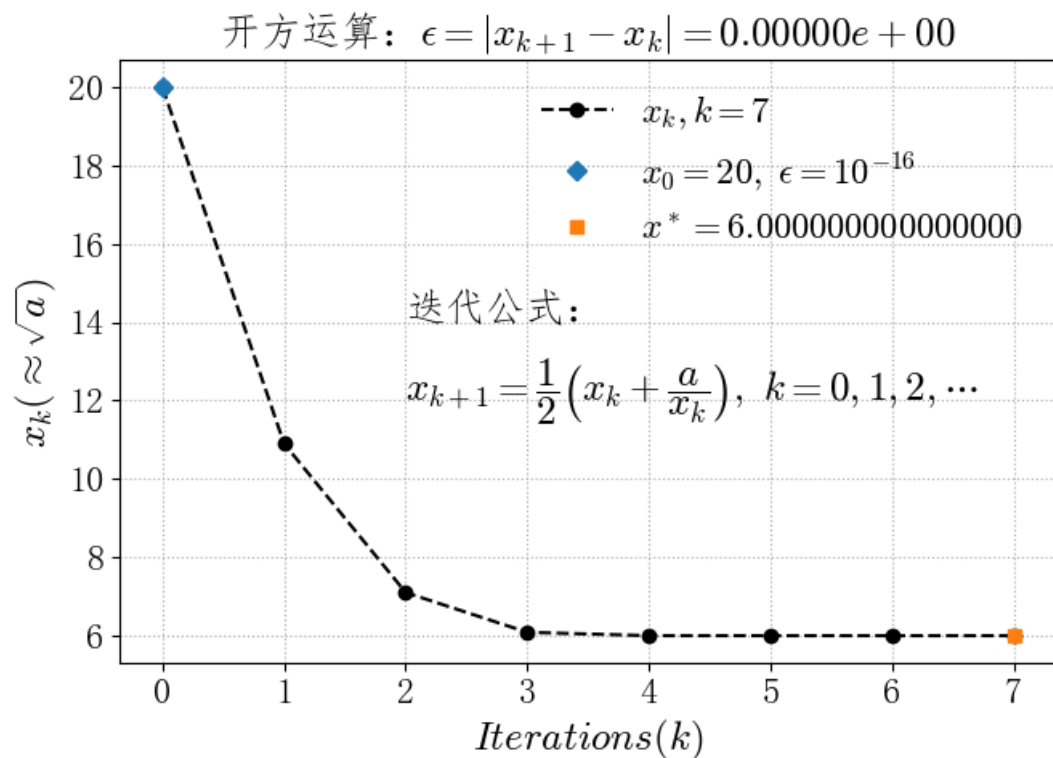
$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right), \quad k = 0, 1, \dots$$

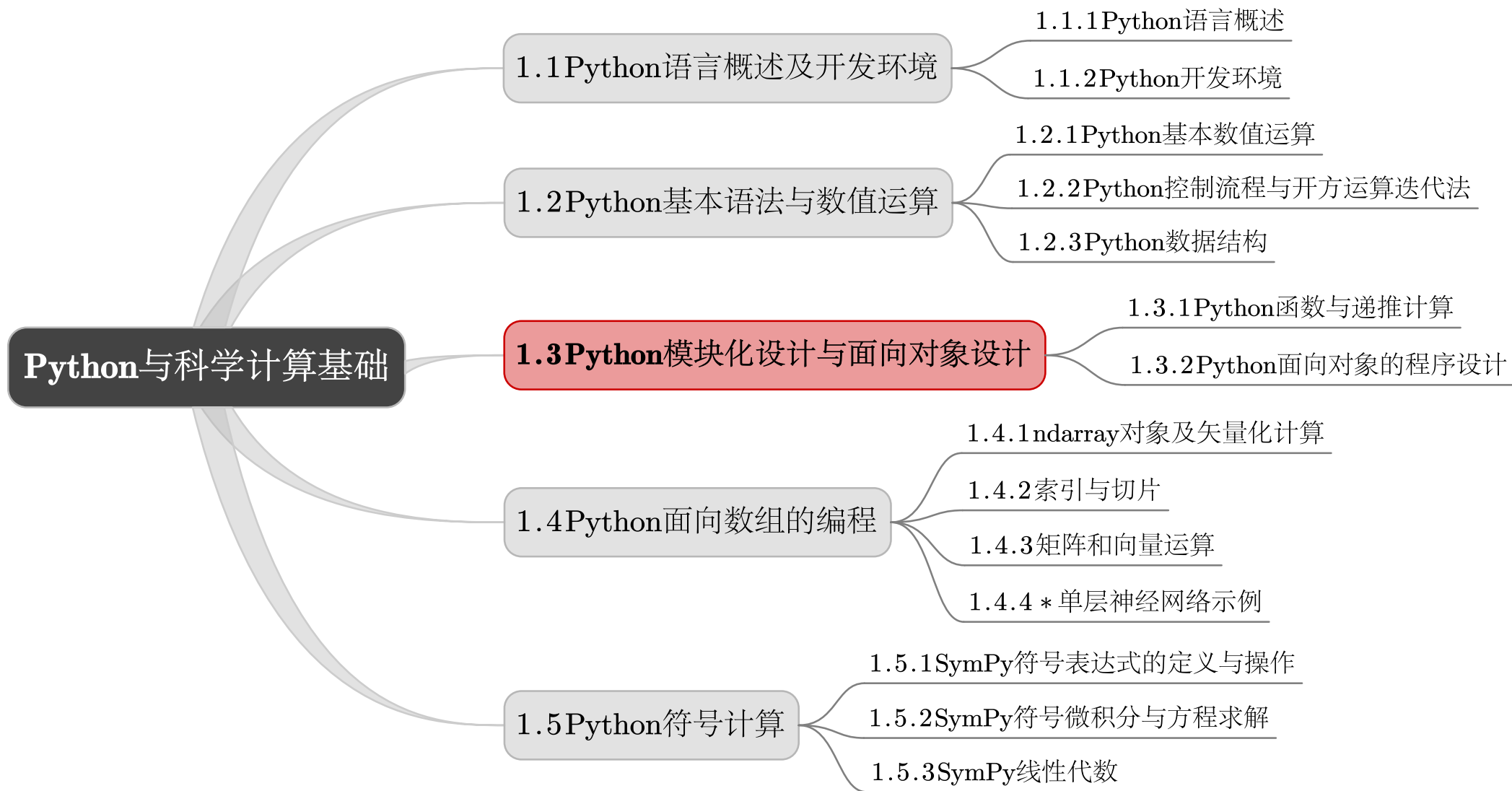
由初值 $x_0 > 0$ 逐步逼近计算, 可得迭代序列 $x_1, x_2, \dots$.

若 $\lim_{k \rightarrow \infty} x_k = x^*$, 则 $x^* = \sqrt{a}$. 对于任意的 $x_0 > 0$, 迭代序列 $\{x_k\}_{k=0}^{\infty}$ 均收敛, 且收敛速度较快.

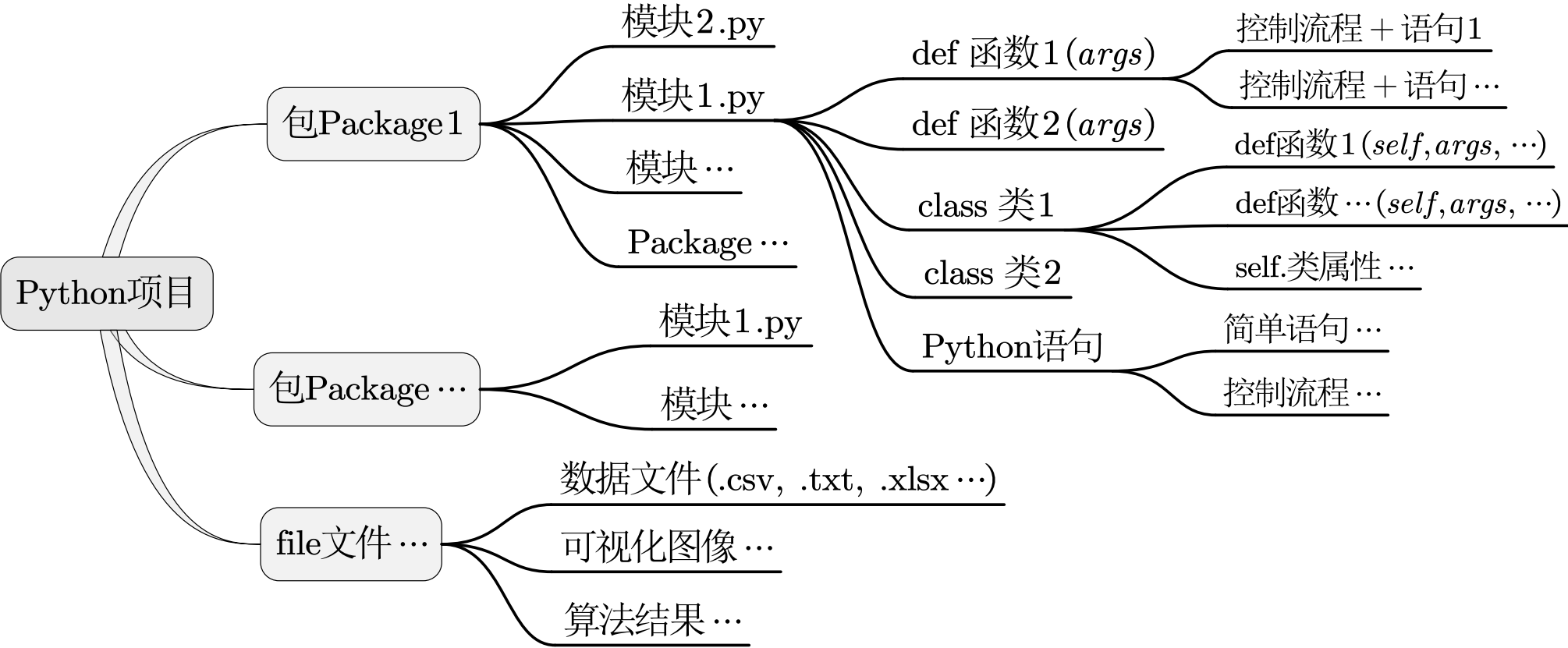
1.2.2 Python控制流程与开方运算迭代法

例1：采用开方运算迭代逼近法求解 $\sqrt{36}$ 和 $\sqrt{\pi}$ ，迭代初值均为 $x_0 = 20$ ，精度要求 $\epsilon = 10^{-16}$ ，最大迭代次数为100。





1.3 Python模块化设计与面向对象设计



1.3.1 Python函数与递推计算

- 声明函数必须使用“**def**”关键字, 且有函数名、形参列表、函数体组成. 如“开方运算”中的函数定义: `def sqrt_cal_while(a, x0, eps=1e-15, max_iter=1000).`
- 匿名函数: **`result = lambda [arg1 [, arg2, ..., argn]] : expreesion`**. 在lambda关键字之后、冒号左边的是参数列表, 可以没有参数, 也可有多个参数; 冒号右边是该lambda 表达式的返回值. 对于单行函数, 使用lambda 表达式可以省去定义函数的过程, 让代码更加简洁. 对于不需要多次复用的函数, 使用lambda 表达式可以在用完之后立即释放, 提高了性能.
- 函数定义时参数列表中的参数就是**形参**, 而函数调用时传递进来的参数则是**实参**. 位置参数也称为**必备参数**, 按照形参位置传入的参数被称为**位置参数**. 如果根据参数名来传入参数值, 则无须遵守定义形参的顺序, 这种方式被称为**关键字参数**.

1.3.1 Python函数与递推计算

➤ **递推计算**是数值分析中常见数值计算方法, 如ODE初边值问题数值解、PDE数值解等.

递推计算由初始条件, 利用递推公式, 逐步递推中间值, 直至得到结果.

➤ **例2:** 模拟飞船软着陆问题

基于自由落体的微分运动方程 $F = ma = m \cdot v'(t) = m \cdot x''(t)$, 转换为一阶微分方程组

$$F = ma = m \frac{dv}{dt} = m \frac{d^2x}{dt^2} \Rightarrow \begin{cases} v'(t) = g, \\ x'(t) = v, \end{cases}$$

其中 m 为物体质量, g 为重力加速度, x 为物体下落的高度, v 为下落速度, t 为时间变量, 忽略单位.

1.3.1 Python函数与递推计算

采用显式欧拉法求解, 可得到微分方程数值解, 其中 h_t 为时间步长, 递推公式为

$$\begin{cases} v(t_{k+1}) = v(t_k) + g \cdot h_t, \\ x(t_{k+1}) = x(t_k) + v(t_{k+1}) \cdot h_t, \end{cases}$$

其中 $t_k = t_0 + kh_t$, $k = 0, 1, \dots$, 递推格式:

$$v(t_0) \rightarrow v(t_1) \rightarrow v(t_2) \rightarrow \dots, \quad x(t_0) \xrightarrow{v(t_1)} x(t_1) \xrightarrow{v(t_2)} x(t_2) \xrightarrow{v(t_3)} \dots.$$

1.3.1 Python函数与递推计算

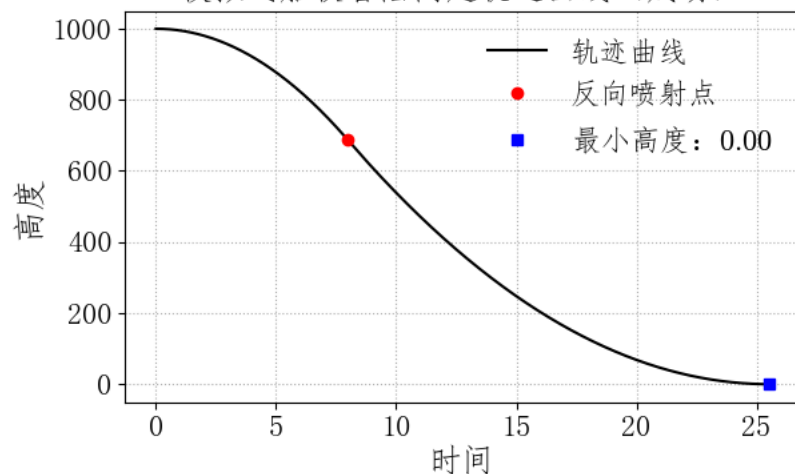
利用自由落体的模拟方法, 模拟反向喷射、软着陆的飞船着陆问题. 反向喷射可以给火箭提供一个向上的加速度, 起到抵消重力加速度的作用. 于是, 具有一定加速度的反向喷射火箭, 其运动方程类似于自由落体方程, 递推公式为

$$\begin{cases} v(t_{k+1}) = v(t_k) + (g - C \cdot a) \cdot h_t, \\ x(t_{k+1}) = x(t_k) + v(t_{k+1}) \cdot h_t, \end{cases}$$

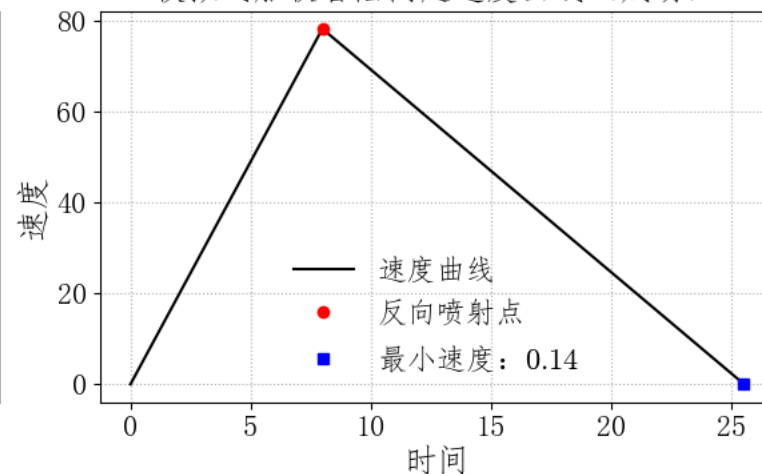
其中 $k = 0, 1, \dots$, C 为反向喷射加速度系数, 为简化计算, 可令为一常数.

1.3.1 Python函数与递推计算

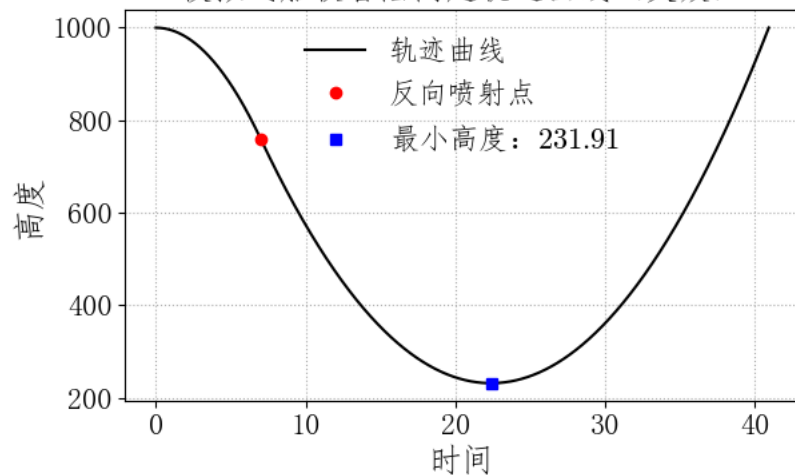
模拟飞船软着陆问题轨迹曲线（成功）



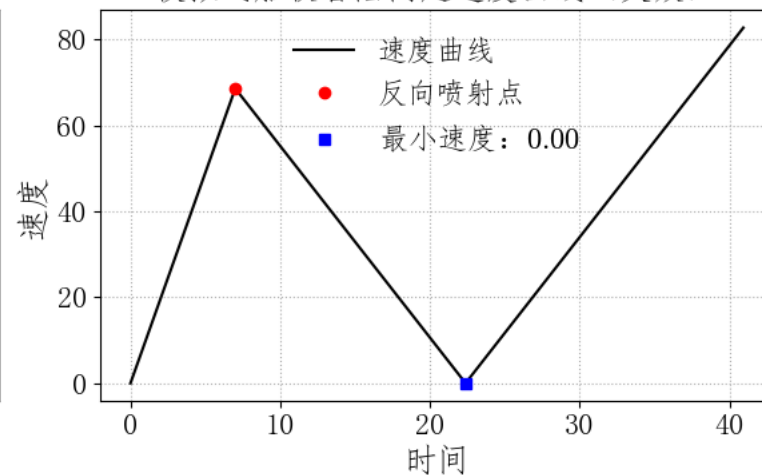
模拟飞船软着陆问题速度曲线（成功）



模拟飞船软着陆问题轨迹曲线（失败）



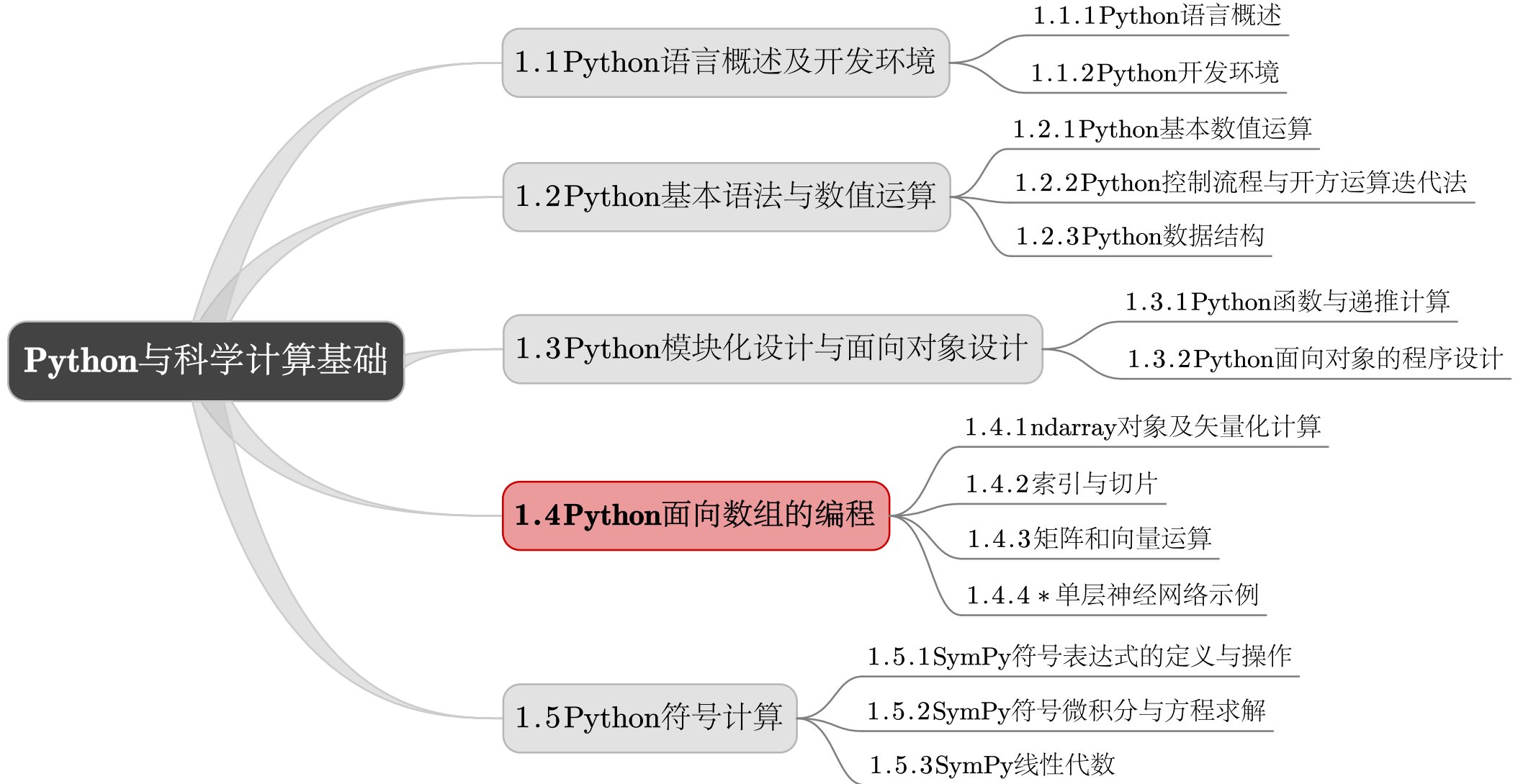
模拟飞船软着陆问题速度曲线（失败）



1.3.2 Python面向对象的程序设计

- 在Python中,“一切皆为对象”.面向对象设计理念是一种从组织结构上模拟客观世界的方法. **类** (Class) 和 **对象** (Object) 是**面向对象编程** (Object Oriented Programming, OOP) 的核心概念:
 - ✓ “类”是封装对象的属性和行为的载体,用于描述一类对象的状态和行为.反过来说,具有相同属性和行为的一类实体被称为类.
 - ✓ “对象”是类的一个实例,实例化的对象有状态 (静态部分) 和行为 (动态部分).
- 类与对象的关系: 类是一类事物的描述,是抽象的. 对象是一类事物的实例,是具体的. 类是对象的模板,对象是类的实体. 类定义了一类对象有哪些属性和方法,但并没有实际的空间,实例化出的对象占有实际空间,用来存储成员变量.

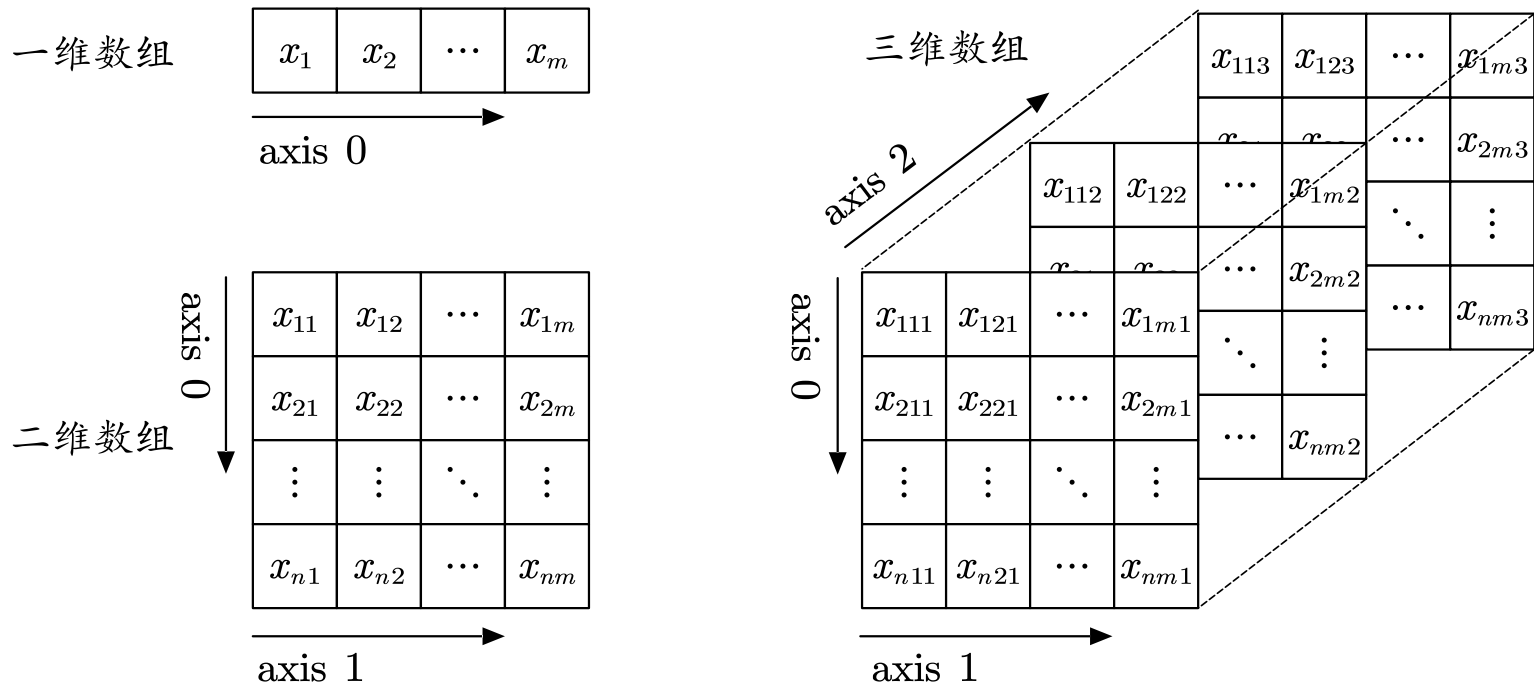
```
12 class SpaceCraftLandingSimulation:
13     """
14     面向对象设计：飞船软着陆模拟计算，即采用显示欧拉法求解一阶微分方程组
15     """
16     G = 9.80665 # 类实例，重力加速度，不同的对象间可共享值
17
18     def __init__(self, init_v, init_h, rf_time, th=0.01, C=1.5):
19         self.init_v = init_v # 初始的速度
20         self.init_h = init_h # 初始的高度
21         self.rf_time = rf_time # 反向喷射开始的时刻
22         self.th = th # 时间跨度，时间步长
23         self.C = C # 反向喷射加速度的系数
24         self.info_dict = dict() # 计算过程信息存储，采用字典，包括时刻、高度和速度，每个均为列表
25         self.rf_ths = None # 为飞船提供反向喷射加速度时的时刻、高度和速度，仅为可视化
26
27     # 控制反向喷射的函数：忽略飞船质量的变化，理想情况下模拟，假设反向喷射加速度a = G
28     retrofire = lambda self, t: -self.C * self.G if t >= self.rf_time else 0.0
29
30     def simulate_cal(self):
31         """
32         核心算法：飞船软着陆模拟计算
33         :return:
34         """
35         height, v, t = self.init_h, self.init_v, 0 # 初始化飞船的高度、速度和时刻
36         self.info_dict["time"] = [t] # 时刻
37         self.info_dict["height"] = [height] # 高度
38         self.info_dict["speed"] = [v] # 速度
39         # 1. 自由落体运动，2. 反向喷射，通过实例函数retrofire(t)控制计算
40         while 0 <= height <= self.init_h: # 循环计算到软着陆或初始的高度为止
41             t += self.th # 更新时刻表
42             v += (self.G + self.retrofire(t)) * self.th # 更新飞船的速度
43             height -= v * self.th # 更新飞船的位置（高度），注意为下降
44             self.info_dict["time"].append(t) # 对应字典的键存储值，每个键所对应的值为列表结构
45             self.info_dict["speed"].append(v) # 速度
46             self.info_dict["height"].append(height) # 高度
47
48         idx = int(self.rf_time / self.th) # 计算反向喷射时刻的索引值，用于后续可视化
49         self.rf_ths = [self.info_dict["time"][idx], self.info_dict["height"][idx], self.info_dict["speed"][idx]]
```



1.4 Python面向数组的编程

- Python面向数组的编程,主要是指基于NumPy库的数值计算. 在Python中, NumPy几乎是一切科学计算的基础库, 是科学计算的基石, 更是当下机器学习、深度学习的基础数值计算库.
- NumPy之于数值计算特别重要的原因之一, 是因为它可以高效处理大数组的数据, 这是list、tuple和dict等无法做到的. NumPy是在一个连续的内存块中存储数据, 独立于其他Python内置对象.
- NumPy核心数据结构是多维数组`ndarray` (N-dimensional array) , ndarray数组是同质的(数据类型相同)、带数据类型的、固定长度的数组. ndarray具有矢量算术运算能力和复杂的广播能力, 并具有执行速度快和节省空间的特点.

1.4 Python面向数组的编程



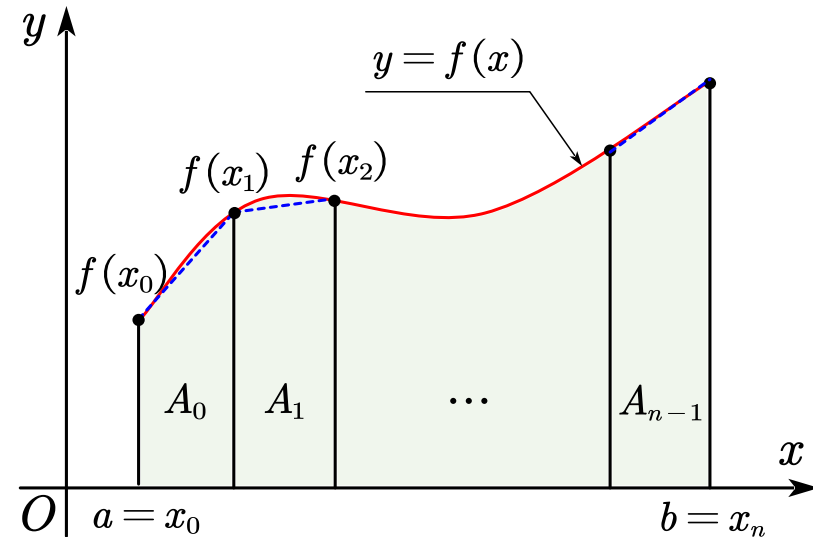
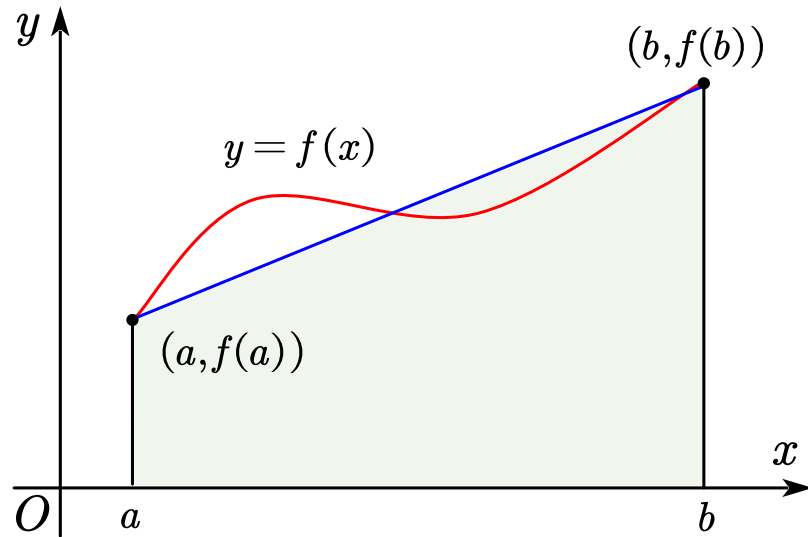
NumPy数组的维数称为秩 (rank), 秩就是轴的数量, 即数组的维度, 一维数组的秩为1, 二维数组的秩为2, 以此类推. 在NumPy中, 每一个线性的数组称为一个轴 (axis), 即维度 (dimensions). 可以声明参数axis, 对于二维数组, axis=0表示沿着第0轴进行操作或运算, 即对每一列进行操作; axis=1表示沿着第1轴进行操作或运算, 即对每一行进行操作.

1.4.1 ndarray对象及矢量化计算

在数值计算中将非线性问题线性化是常用方法. 如下图（左）所示, 计算定积分的梯形公式

$$I(f) = \int_a^b f(x) dx \approx \frac{b-a}{2} [f(a) + f(b)]$$

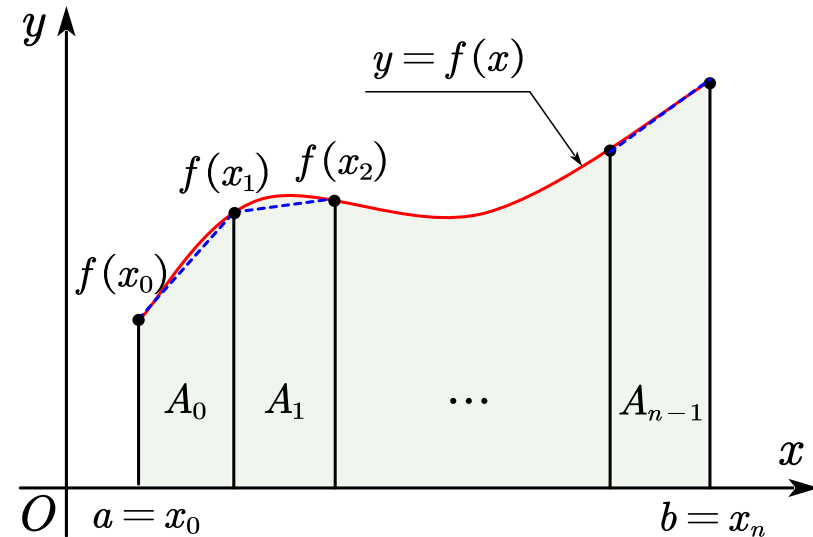
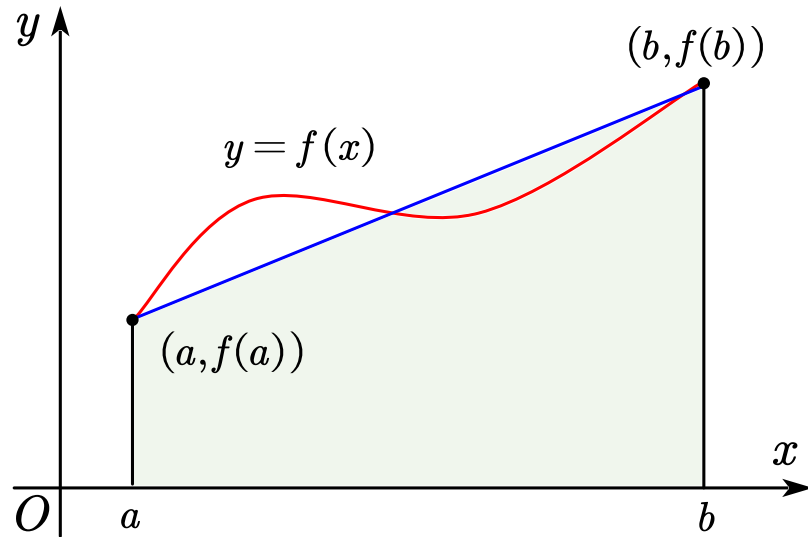
它是用通过曲线上两点 $(a, f(a))$ 、 $(b, f(b))$ 的直线近似曲线的弧, 用梯形面积近似曲边梯形面积, 即“以直带曲”.



1.4.1 ndarray对象及矢量化计算

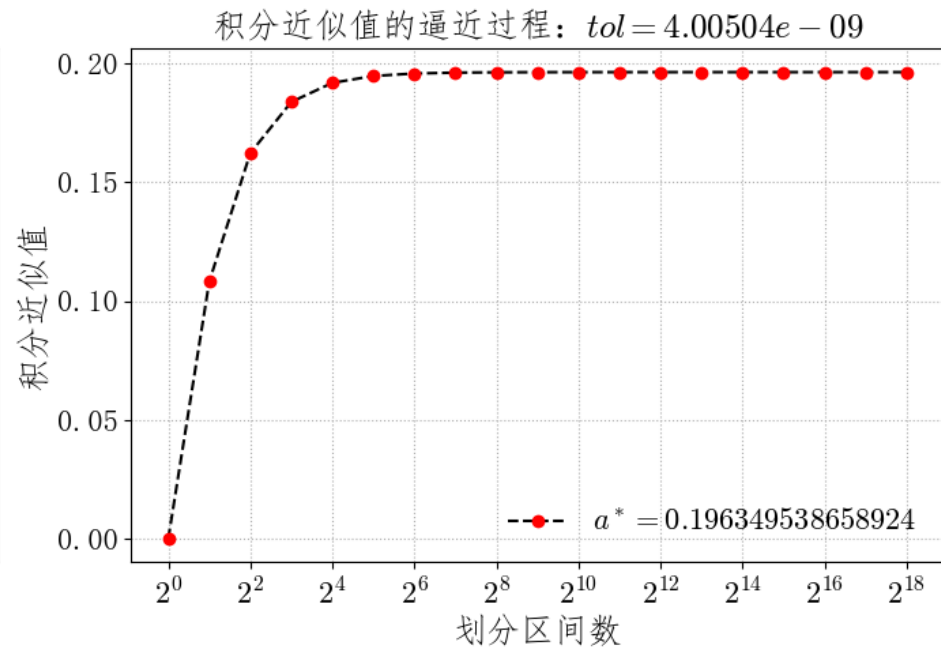
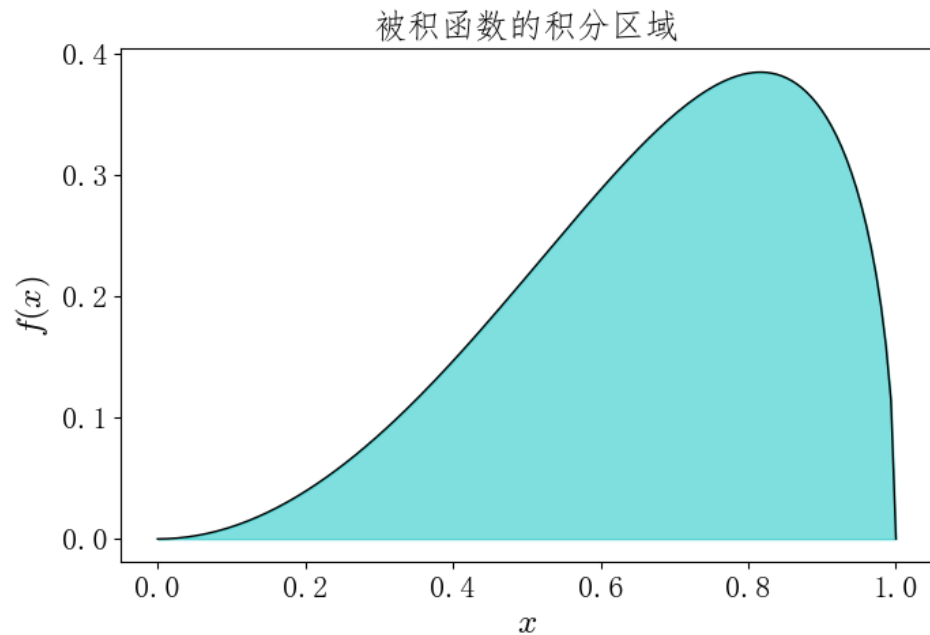
为提高计算精度采用化整为“零”思想，将 $[a, b]$ 分割为小区间 $a = x_0 < x_1 < \cdots < x_n = b$ ，其中 $x_i = a + ih, h = (b - a)/n$ ，在每个小区间 $[x_i, x_{i+1}]$, $i = 0, 1, \cdots, n - 1$ 上用梯形公式计算，再求和得到

$$I(f) = \int_a^b f(x) dx \approx \sum_{i=0}^{n-1} \frac{h}{2} (f(x_i) + f(x_{i+1}))$$



1.4.1 ndarray对象及矢量化计算

例3: 计算 $I(f) = \int_0^1 x^2 \sqrt{1-x^2} dx = \frac{\pi}{16}$, 精度要求 $\varepsilon = 10^{-8}$, 并对比矢量化和非矢量化计算的效率.

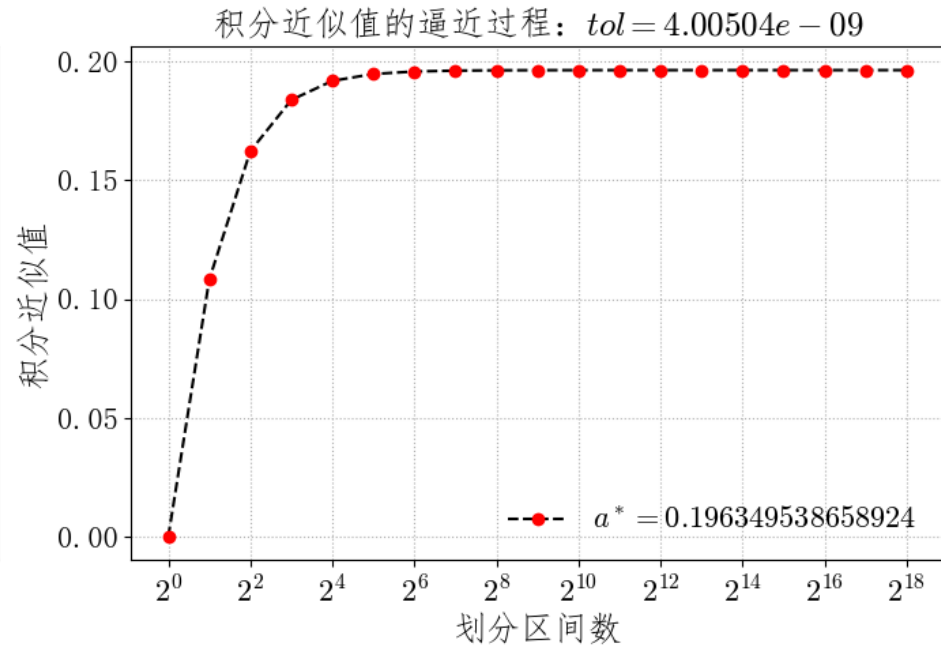
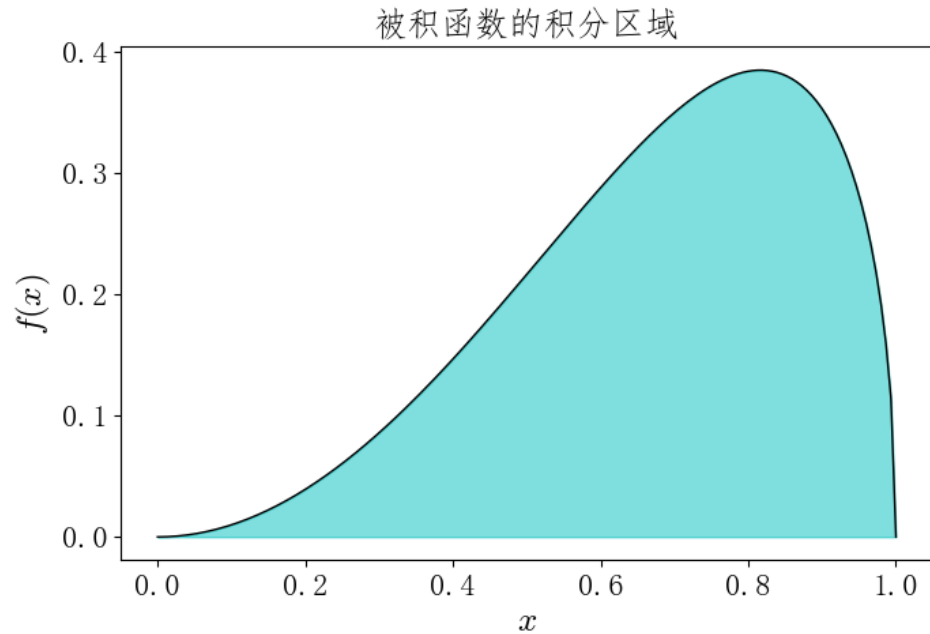


矢量化计算和非矢量化计算计算结果一致. 最终划分区间数262144.

近似积分值为 $I^* \approx 0.196349538658924$, 绝对值误差为 $2.1904350278045825 \times 10^{-9}$.

1.4.1 ndarray对象及矢量化计算

例3: 计算 $I(f) = \int_0^1 x^2 \sqrt{1-x^2} dx = \frac{\pi}{16}$, 精度要求 $\varepsilon = 10^{-8}$, 并对比矢量化和非矢量化计算的效率.



循环计算100次, 统计矢量化和非矢量化计算的时间(假设忽略其他影响计算消耗时间的客观因素), 得其平均时间分别为 $\bar{t}_1 = 0.010434315204620362$ 和 $\bar{t}_2 = 0.8909119415283203$ (单位:秒), 比值约为 $\bar{t}_2/\bar{t}_1 \approx 85.383$, 可见矢量化计算的执行效率远高于非矢量化计算的效率.

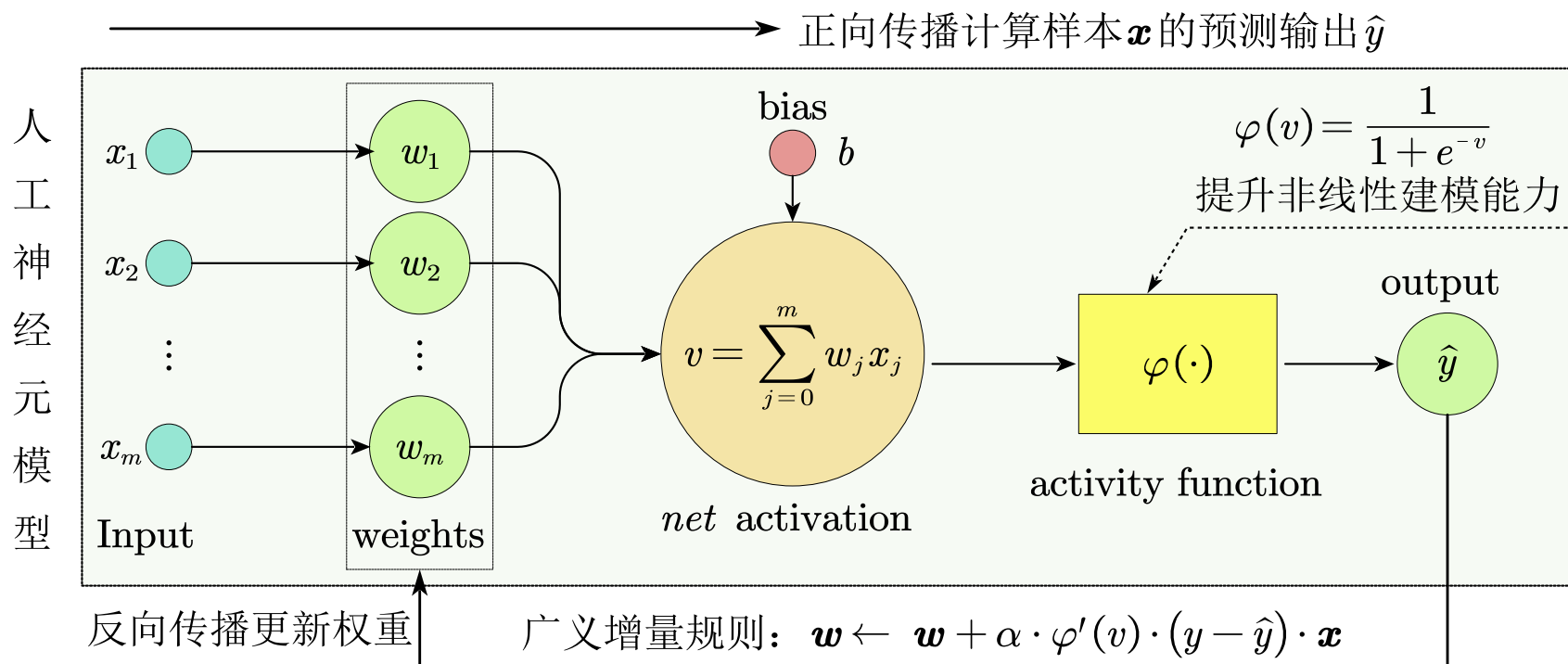
1.4.3 矩阵和向量运算

- **广播 (Broadcast)** 是NumPy对不同形状 (shape) 的数组进行数值计算的方式, 对数组的算术运算通常在相应的元素上进行. 具体如下:
 - ✓ 向量和标量运算, 标量自动复制扩充为向量;
 - ✓ 矩阵和向量运算, 向量在没对齐的维度自动复制扩充为矩阵;
 - ✓ 两个矩阵在两个维度上都不对齐, 且都能广播, 在每个维度上分别复制扩充.
- 广播的原则: 如果两个数组的后缘维度 (trailing dimension, 即从末尾开始算起的维度) 的轴长度相符, 或其中的一方的长度为1, 则认为它们是广播兼容的. 广播会在缺失和 (或) 长度为1的维度上进行.

1.4.4 *单层神经网络示例

下图为人工神经元模型. 函数 $\varphi(\cdot)$ 称为激活函数或转移函数(transfer Function), net 称为净激活(net activation). 神经元的输出与输入的关系表示为

$$net_i = \sum_{j=0}^m w_j x_{ij} = \mathbf{w}^T \mathbf{x}_i, \hat{y}_i = \varphi(net_i), i = 1, 2, \dots, n.$$



1.4.4 *单层神经网络示例

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix}, \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{pmatrix}$$

用适当的值初始化权重, 则单层神经网络的训练过程(针对单个样本而言):

- (1)前向传播: 从训练数据中获得“输入” $\mathbf{x}_i = (x_{i1}, x_{i2}, \cdots, x_{im})^T$, 将“输入”传递到神经网络模型中, 从模型获得预测输出 $\hat{y}_i = \varphi(\mathbf{w}^T \mathbf{x}_i)$, $i = 1, 2, \cdots, n$;
- (2)反向传播: 依据“正确输出” y_i 计算误差 $e_i = y_i - \hat{y}_i$, 按照增量规则计算权重的更新, 即 $\Delta \mathbf{w} = \alpha e_i \mathbf{x}_i$, α 为学习率, 调整权重 $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w} = \mathbf{w} + \alpha e_i \mathbf{x}_i, i = 1, 2, \cdots, n$;
- (3)将所有训练数据重复执行第(1)、(2)步, 直至满足某种收敛性条件, 停止训练.

1.4.4 *单层神经网络示例

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1m} \\ x_{21} & x_{22} & \cdots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{nm} \end{pmatrix}, \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{pmatrix}$$

用适当的值初始化权重, 则单层神经网络的训练过程(针对单个样本而言):

- (1)前向传播: 从训练数据中获得“输入” $\mathbf{x}_i = (x_{i1}, x_{i2}, \cdots, x_{im})^T$, 将“输入”传递到神经网络模型中, 从模型获得预测输出 $\hat{y}_i = \varphi(\mathbf{w}^T \mathbf{x}_i)$, $i = 1, 2, \cdots, n$;
- (2)反向传播: 依据“正确输出” y_i 计算误差 $e_i = y_i - \hat{y}_i$, 按照增量规则计算权重的更新, 即 $\Delta \mathbf{w} = \alpha e_i \mathbf{x}_i$, α 为学习率, 调整权重 $\mathbf{w} \leftarrow \mathbf{w} + \Delta \mathbf{w} = \mathbf{w} + \alpha e_i \mathbf{x}_i, i = 1, 2, \cdots, n$;
- (3)将所有训练数据重复执行第(1)、(2)步, 直至满足某种收敛性条件, 停止训练.

1.4.4 *单层神经网络示例

例4: 分别以sklearn 库数据集“鸢尾花Iris”和“乳腺癌Breast_Cancer”为训练样本集和测试样本集, 划分比例为 . 以学习率、交叉熵损失精度 和最大训练次数 训练网络.

(1) 鸢尾花Iris数据集, 共150个样本, 3个类别, 每个样本4个特征变量, 分别为花萼长度 (Sepal Length, SL)、花萼宽度 (Sepal Width, SW)、花瓣长度 (Petal Length, PL) 和花瓣宽度 (Petal Width, PW) . 取前100个训练样本, 分别构成两个类别山鸢尾 (Setosa) 和变色鸢尾 (Versicolor) , 每个类别50个样本.

测试样本预测正确率分别为100%. 鸢尾花数据集最终训练的网络权重为

$$\mathbf{w} = (0.59640037, -0.90132166, 1.10391158, 1.12732193)^T,$$

则单层神经网络模型为

$$\hat{y} = \varphi(\mathbf{w}^T \mathbf{x}) = \varphi(0.59640037x_1 - 0.90132166x_2 + 1.10391158x_3 + 1.12732193x_4).$$

1.4.4 *单层神经网络示例

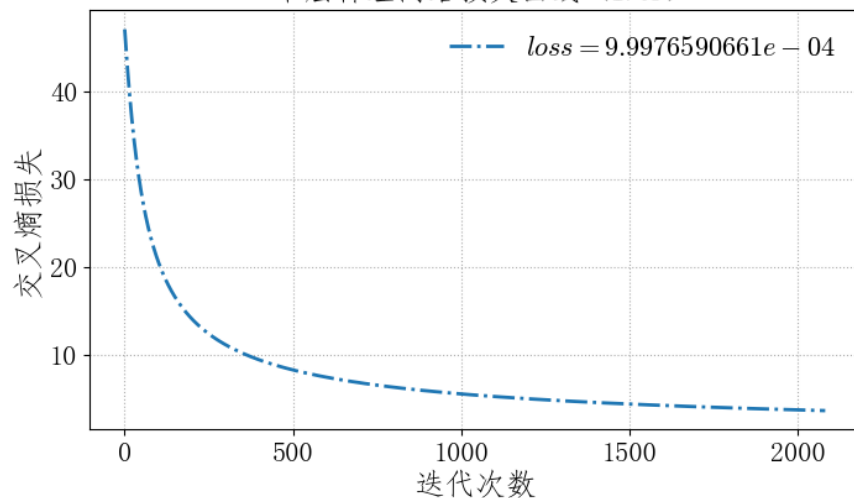
(2) 乳腺癌Breast_Cancer , 共包含569个样本, 每个样本30个特征变量, 两个类别分别为恶性malignant和良性benign.

测试样本预测正确率分别为96.5%. 单层神经网络模型为

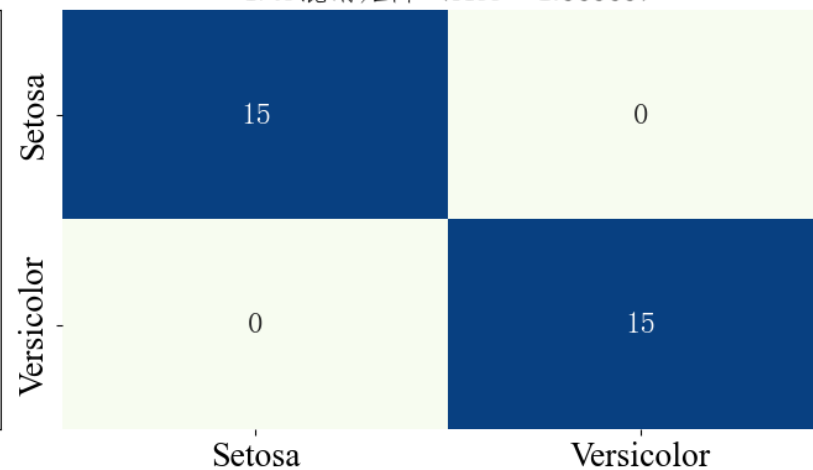
$$\hat{y} = \varphi(\mathbf{w}^T \mathbf{x}) = \varphi(-0.48062351x_1 - 0.43015921x_2 + \cdots - 0.26220939x_{30}).$$

1.4.4 *单层神经网络示例

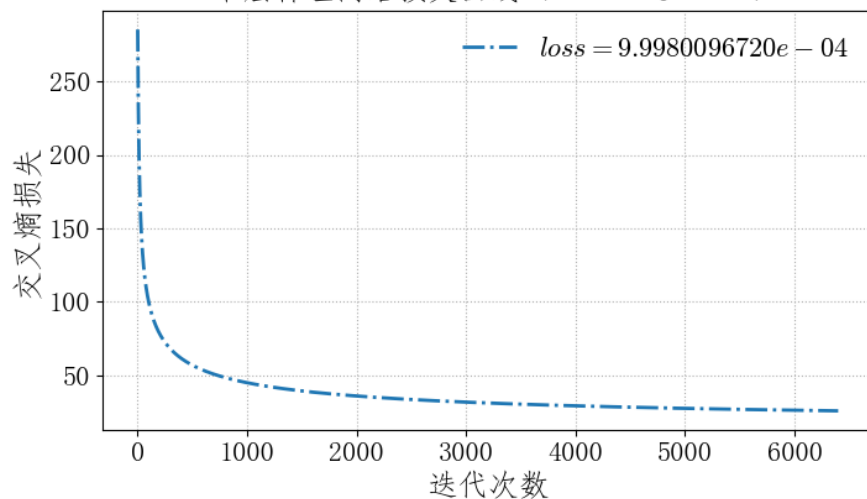
单层神经网络损失曲线 (Iris)



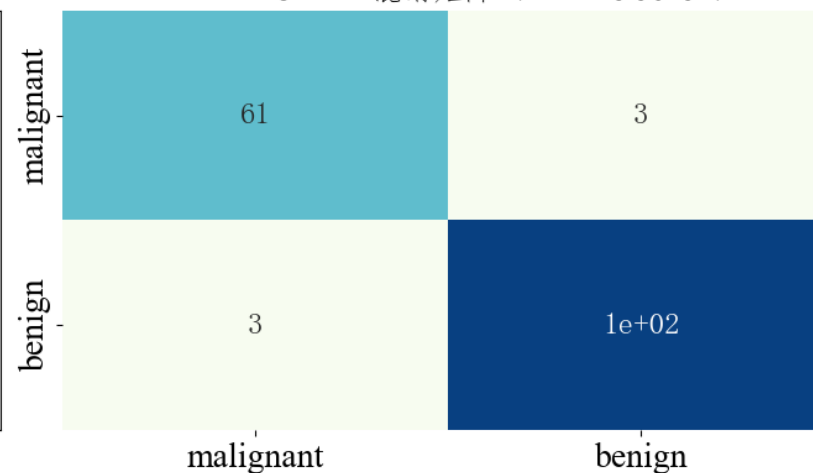
Iris混淆矩阵 (Acc = 1.00000)

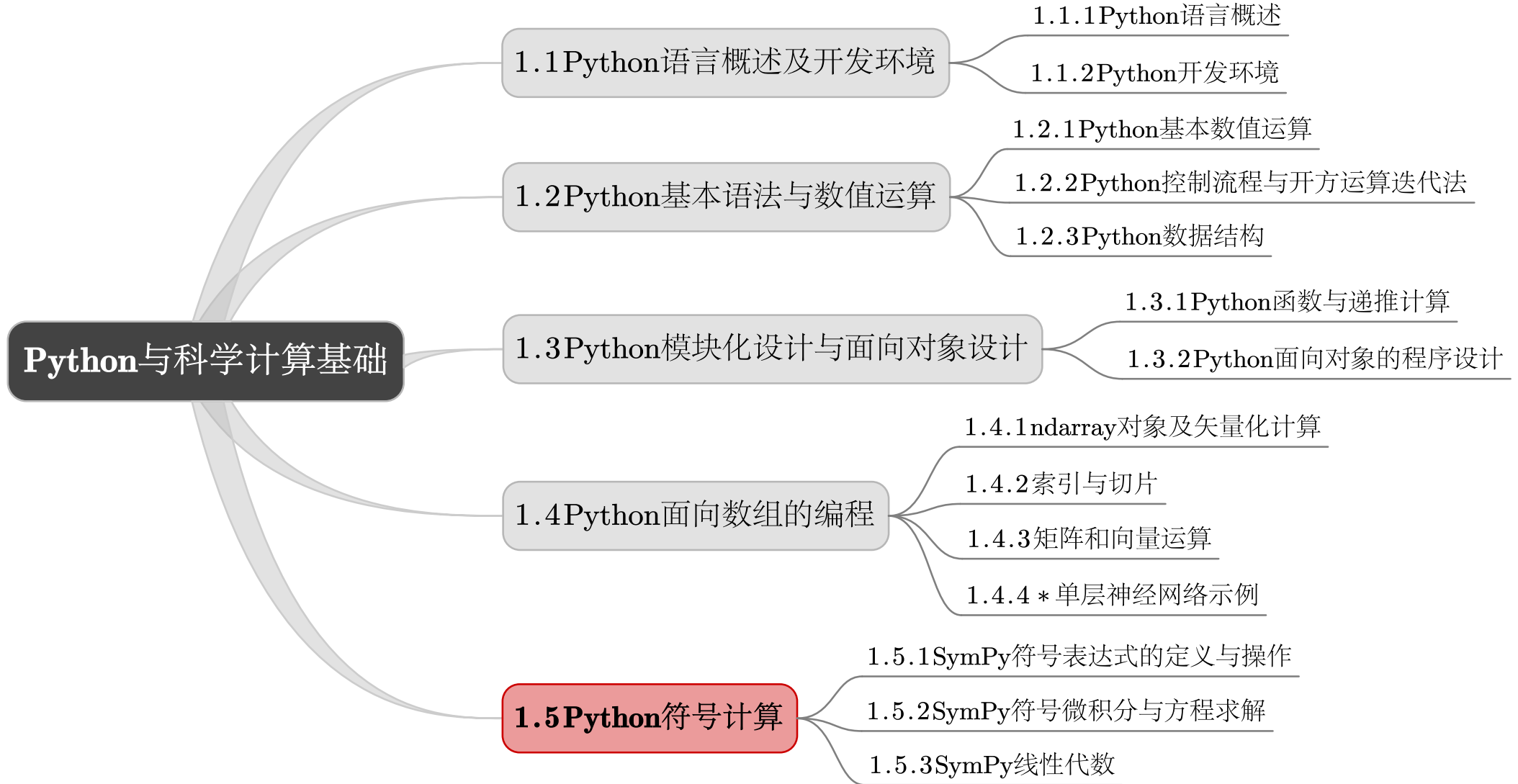


单层神经网络损失曲线 (Breast Cancer)



Breast Cancer混淆矩阵 (Acc = 0.96491)





1.5 Python符号计算

在Python中, 有关数学类的运算, 可分为数值计算和符号计算. 自然科学理论分析中的公式、关系式及其推导是符号计算要解决的问题. 因而, 所谓符号计算就是基于数学公式、定理并通过一系列推理、演绎得到方程的解或表达式的值. 符号计算对操作对象不进行离散化和近似化处理, 故可得到问题精确的完备解, 但是计算量大且表达形式庞大.

典型的符号运算包括: (1) 表达式化简、求值, 以及变形为展开、积、幂、部分分式表示等; (2) 一元或多元函数微分, 表达式或函数极限, 函数的定积分与不定积分, Taylor展开, 无穷级数展开, 级数求和; (3) 求解线性或非线性方程问题, 求解微分方程或差分方程问题; (4) 矩阵运算等.

SymPy是Python的一个数学符号计算库, 它的目标是成为一个全功能的计算机代数系统, 同时保持代码的精简而易于理解和可扩展. 在使用SymPy之前需要先将其导入: `import sympy`.

1.5.1 SymPy符号表达式的定义与操作

表 1-11 SymPy 常见符号表达式操作函数

函数名称	说明
<code>sympy.Lambda</code>	该函数将一组自由符号和一个表达式作为参数, 生成一个能对表达式进行高效数值计算的函数. 所生成函数的参数数量与传给 <code>sympy.lambdify</code> 的自由符号数量一样.
<code>sympy.simplify</code>	自行寻找它认为的最简单的表达形式 (尝试各种方法对表达式进行化简), 也可通过调用表达式的 <code>simplify</code> 方法来化简.
<code>expr.subs</code> , <code>expr.replace</code>	进行替换操作. <code>subs</code> 函数是最合适的选择, 但在某些情况下, <code>replace</code> 函数能够提供更强大的功能, 如使用通配符表达式进行替换. 当需要进行多个替换时, 不需要把多个 <code>subs</code> 函数连起来, 而只需要传递一个字典类型的参数给 <code>subs</code> 函数, 再在字典中把旧的符号或表达式映射到新的符号或表达式.
<code>sympy.N</code> , <code>expr.evalf</code>	对表达式进行求值, 使用一个可选参数来指定要计算的表达式的有效位数, SymPy 的多精度浮点函数能够计算高达 50 位的 π 值.

1.5.2 SymPy符号微积分与方程求解

表 1-12 SymPy 符号微积分与符号方程求解函数

函数名称	说明
<code>sympy.diff</code> , <code>expr.diff</code>	计算函数导数. 直接对表达式调用 <code>sympy.diff</code> 会产生一个新的表达式.
<code>sympy.integrate</code>	求定积分和不定积分, 传递元组参数 (x, a, b) 则意味着计算定积分. 注 未必得到解析解
<code>sympy.series</code> , <code>expr.series</code>	级数展开. <code>sympy.series</code> 第 1 个参数为函数或表达式, 第 2 个参数是要计算展开的符号; 可指定围绕某个点执行展开以及展开的阶数.
<code>sympy.limit</code>	函数或表达式的极限. 输入参数包括求极限的表达式、变量符号以及自变量靠近的点.
<code>sympy.Sum</code> , <code>sympy.Product</code>	表示和与积. 当 <code>sympy.Sum</code> 和 <code>sympy.Product</code> 对象创建之后, 可以使用 <code>doit</code> 方法来求值.
<code>sympy.solve</code>	求解方程 (组). 注 未必得到解析解.

1.5.3 SymPy线性代数

表 1-13 SymPy 符号线性代数操作函数或方法

函数/方法	说明	函数/方法	说明
transpose / T	计算矩阵的转置矩阵.	QRdecomposition	计算矩阵的 QR 分解.
adjoint / H	计算矩阵的伴随矩阵.	QRsolve	使用 QR 分解求线性方程组 $Ax = b$ 的解.
trace	计算矩阵的迹 (主对角线上元素的和).	diagonalize	矩阵对角化, 写成 $D = P^{-1}AP$ 的形式, 其中 D 是对角矩阵.
det	计算矩阵的行列式.	norm	计算矩阵的范数.
inv	计算矩阵的逆矩阵.	nullspace	计算矩阵的零空间.
LUdecomposition	计算矩阵的 LU 分解.	rank	计算矩阵的等级, 即秩.
LUsolve	使用 LU 分解求线性方程组 $Ax = b$ 的解.	singular_values	计算矩阵的奇异值.
solve	求线性方程组 $Ax = b$ 的解.	eigenvals, eigenvects	计算矩阵的特征值和特征向量.